ShadowMem — Codebase Errors & Actionable Rectification Guide

Technical Root Causes, Reproduction Failures, and Copy-Paste Ready Fixes

Target Repository: c:\Users\Himesh\Desktop\ShadowMem | **Audit Date:** 2026-10-31 14:08 | **Scope:** Rectification Guide Only

This engineering report outlines all diagnosed bugs, runtime exceptions, handle leaks, test suite logic errors, and architectural inconsistencies identified in the ShadowMem repository. Each item includes the exact target file, line numbers, root cause explanation, exception trace, and copy-paste ready rectified code.

1. Executive Actionable Errors Matrix

ID	Severity	Target File & Lines	Issue Description	Runtime Impact
ERR-01	**CRITICAL**	app.py 388, 406, 477, 534, 599, 603, 642, 694, 757, 761	wintypes.INVALID_HANDLE_VALUE & NTSTATUS AttributeError. Neither attribute exists in ctypes.wintypes.	Isolate/Resume crashes on real PIDs; handle cleanup in finally blocks fails silently, causing permanent kernel handle leaks.
ERR-02	**CRITICAL**	app.py 386	for tid in proc.num_threads(): iterates over an integer instead of thread objects.	Raises TypeError: 'int' object is not iterable on every containment call; breaks primary thread enumeration.
ERR-03	**HIGH**	app.py 2123	CSS class 'pointer-events-none' added to modalIsolateBtn when process is suspended.	Completely disables mouse clicks on 'Resume Process' button; operator is permanently locked out from resuming processes.
ERR-04	**HIGH**	test_sandbox.py 1-5, 185, 249-263	Variable shadowing 'for _, passed in results', dict.get() NoneType AttributeError, and OpenMP thread explosion.	Test runner reports '5 TESTS FAILED' and exits code 1 even when all tests pass; Test 6 crashes on clean scans.
ERR-05	**MEDIUM**	test_fixes.py: 6 malware_simulator.py: 366	Hardcoded timeout=30 in urllib.request.urlopen for /api/scan endpoint.	Full memory scan of 120+ processes takes ~33-36s; client aborts prematurely with TimeoutError: timed out.
ERR-06	**MEDIUM**	malware_simulator.py 104, 107, 341	REFRESH_INTERVAL=5.0 triggers overlapping 33s scans; calls missing route /simulate/alert on app.py.	Pins CPU at 100% due to accumulated concurrent scans; prints continuous HTTP 404 errors.
ERR-07	**MEDIUM**	detecter.py: 118, 131 make_guide.py: 246	Windows Console UnicodeEncodeError on checkmark character (\u2713 / ✓) in cp1252.	CLI detection script and guide builder crash with UnicodeEncodeError upon completion on default Windows terminal.
ERR-08	**LOW**	test_injector.py 298-302	Missing kernel32.CloseHandle(spin_thread) after thread termination in finally block.	Open kernel thread handle is leaked during test execution until python process termination.
ERR-09	**MEDIUM**	server.py vs app.py index.html, trigger_test_event.py	Two disconnected server architectures: standalone mock WebSocket server vs real Win32 forensics engine.	trigger_test_event.py and standalone index.html fail when launch_shadowmem.bat starts app.py.

2. Technical Breakdown & Copy-Paste Ready Code Rectifications

ERR-01 [CRITICAL]: Invalid wintypes.INVALID_HANDLE_VALUE & NTSTATUS AttributeError Crash + Handle Leaks

Target File: app.py (Lines 388, 406, 477, 534, 599, 603, 642, 694, 757, 761)

Root Cause: Python's ctypes.wintypes module does NOT define INVALID_HANDLE_VALUE or NTSTATUS. When a valid process or thread handle is acquired, testing handle == wintypes.INVALID_HANDLE_VALUE immediately raises AttributeError: module 'ctypes.wintypes' has no attribute 'INVALID_HANDLE_VALUE'. In lines 599, 603, 757, and 761 (inside the finally: block), this error is swallowed by except Exception: pass, causing _CLOSE_HANDLE(h) to NEVER execute, permanently leaking kernel handles! Furthermore, lines 534 and 694 set .restype = wintypes.NTSTATUS which crashes, and improperly evaluate if status <= 0: (in NTSTATUS, 0 is STATUS_SUCCESS, so success was treated as failure).

Reproduction Trace:

```
Traceback (most recent call last):
File "app.py", line 477, in api_isolate
if not handle or handle == wintypes.INVALID_HANDLE_VALUE:
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AttributeError: module 'ctypes.wintypes' has no attribute 'INVALID_HANDLE_VALUE'
```

Rectified Code for app.py:

```
# 1. In app.py around line 301, define INVALID_HANDLE_VALUE:
from ctypes import wintypes
INVALID_HANDLE_VALUE = -1 # or ctypes.c_void_p(-1).value

# 2. In lines 477 and 642, check handle validity safely:
handle = _OPEN_PROCESS(PROCESS_SUSPEND_RESUME, False, pid)
if not handle or handle == INVALID_HANDLE_VALUE:
err = _K32.GetLastError()
# ... error handling ...

# 3. In finally blocks (lines 598-605, 755-765), clean up handles without crashing:
finally:
for h in handles:
try:
if h and h != INVALID_HANDLE_VALUE:
_CLOSE_HANDLE(h)
except Exception:
pass
if handle and handle != INVALID_HANDLE_VALUE:
try:
_CLOSE_HANDLE(handle)
except Exception:
pass

# 4. In lines 533-536 and 693-696, fix NTSTATUS signature and check:
_NtSuspendProcess.argtypes = (wintypes.HANDLE,)
_NtSuspendProcess.restype = ctypes.c_long # Use c_long instead of wintypes.NTSTATUS
status = _NtSuspendProcess(handle)
if status != 0: # 0 is STATUS_SUCCESS in NTSTATUS
pass
else:
suspended = max(suspended, 1)
```

ERR-02 [CRITICAL]: TypeError on Thread Enumeration (proc.num_threads() in for loop)

Target File: app.py (Line 386 in _iter_threads)

Root Cause: Line 386 attempts to iterate over proc.num_threads(): for tid in proc.num_threads():. The psutil method proc.num_threads() returns a single integer (e.g., 14). In Python, integers are not iterable. This triggers an immediate TypeError on every call, completely breaking primary thread enumeration.

Reproduction Trace:

```
TypeError: 'int' object is not iterable
File "app.py", line 386, in _iter_threads
for tid in proc.num_threads():
```

Rectified Code for app.py (lines 383-394):

```
if _psutil is not None:
try:
proc = _psutil.Process(pid)
for td in proc.threads(): # Iterate over thread objects, NOT proc.num_threads()
h = _OPEN_THREAD(THREAD_SUSPEND_RESUME, False, td.id)
if h and h != INVALID_HANDLE_VALUE:
yield td.id, h
continue
except Exception:
pass
```

ERR-03 [HIGH]: Process Containment Button UI Lockout (app.py)

Target File: app.py (Line 2123 in setIsolateUI)

Root Cause: When an operator suspends an anomalous process from the web dashboard, setIsolateUI(pid, true) executes. Line 2123 applies the Tailwind CSS utility class pointer-events-none to modalIsolateBtn while changing its label to 'Resume Process'. The pointer-events-none rule disables all mouse click events. As a result, the operator is permanently locked out from clicking 'Resume Process' until they refresh the browser page.

Rectified Code for app.py (lines 2119-2139):

```
function setIsolateUI(pid, suspended) {
if (suspended) {
isolatedPids.add(pid);
modalIsolatedBadge.classList.remove('hidden');
modalIsolateBtn.classList.remove('pointer-events-none'); // DO NOT disable pointer-events
modalIsolateBtn.innerHTML =
'<svg class="w-3.5 h-3.5 text-ok" fill="none" viewBox="0 0 24 24" stroke="currentColor" stroke-width="2">'
+ '<path stroke-linecap="round" stroke-linejoin="round" d="M5 13l4 4L19 7" />'
+ '</svg>'
+ '<span>Resume Process</span>';
} else {
isolatedPids.delete(pid);
modalIsolatedBadge.classList.add('hidden');
modalIsolateBtn.classList.remove('opacity-50', 'pointer-events-none');
modalIsolateBtn.innerHTML =
'<svg class="w-3.5 h-3.5 text-warn" fill="none" viewBox="0 0 24 24" stroke="currentColor" stroke-width="2">'
+ '<path stroke-linecap="round" stroke-linejoin="round" d="M18.364 18.364A9 9 0 005.636 5.636m12.728 12.728A9 9 0 015.636
5.636m12.728 12.728L5.636 5.636" />'
+ '</svg>'
+ '<span>Isolate Process</span>';
}
}
```

ERR-04 [HIGH]: Variable Shadowing, AttributeError & Thread Count Inflation (test_sandbox.py)

Target File: test_sandbox.py (Lines 1-5, 185, 249-263)

Root Cause 1 (Shadowing): In main(), line 249 calculates passed = sum(...). Line 252 reuses passed: for test_name, passed in results:. This overwrites the integer tally with a boolean (True). At line 258, if passed == total: evaluates True == 6 (False), reporting '*** 5 TESTS FAILED' and exiting with code 1.
Root Cause 2 (NoneType): In Test 6, result.get('classification', {}) returns None because the key exists with a None value. Calling classification.get(...) raises AttributeError: 'NoneType' object has no attribute 'get'.
Root Cause 3 (Thread Accumulation): Running tests sequentially accumulates NumPy/OpenMP worker threads up to 45 threads in the runner PID. The ML model treats processes with >35 threads as benign, suppressing the threat score to 9% and setting classification: None.

Rectified Code for test_sandbox.py:

```
# 1. Add at top of test_sandbox.py (before importing numpy/sklearn/app):
import os
os.environ['OMP_NUM_THREADS'] = '1'
os.environ['OPENBLAS_NUM_THREADS'] = '1'
os.environ['MKL_NUM_THREADS'] = '1'

# 2. In line 185 (test_technique_classification):
classification = result.get('classification') or {}

# 3. In lines 249-263 (fix shadowing):
passed_count = sum(1 for _, p in results if p)
total_count = len(results)
for test_name, test_status in results: # Renamed loop variable to avoid shadowing
status = 'PASS' if test_status else 'FAIL'
print(f' [{status}] {test_name}')
print(f'\nTotal: {passed_count}/{total_count} test suites passed')
if passed_count == total_count:
print('\n*** ALL TESTS PASSED - Sandbox is working correctly ***')
return 0
else:
print(f'\n*** {total_count - passed_count} TESTS FAILED - Review needed ***')
return 1
```

ERR-05 [MEDIUM]: Premature Socket Timeout on Full Memory Scans (test_fixes.py & malware_simulator.py)

Target Files: test_fixes.py (Line 6), malware_simulator.py (Line 366)

Root Cause: Both files configure timeout=30 in urllib.request.urlopen. On a live Windows 11 system with 120-150 active processes, walking all process address spaces via Win32 VirtualQueryEx and NtQueryInformationThread requires approximately 33 to 36 seconds. The client triggers a premature TimeoutError: timed out even though the backend API is working normally.

Rectified Code:

```
# In test_fixes.py line 6:
r = urllib.request.urlopen('http://127.0.0.1:8000/api/scan', timeout=60)

# In malware_simulator.py line 366:
with urllib.request.urlopen(req, timeout=60) as resp:
```

ERR-06 [MEDIUM]: Scan Flooding (5s Interval) & Missing /simulate/alert Route (malware_simulator.py)

Target File: malware_simulator.py (Lines 104, 107, 341-357)

Root Cause: REFRESH_INTERVAL = 5.0 spawns a scan every 5 seconds. Because full memory scans take ~33s, 6 to 7 scans accumulate concurrently, driving CPU usage to 100% and starving the Uvicorn server. Additionally, DISPLAY_MODE = 'both' attempts to POST to /simulate/alert, which does not exist on app.py, spitting out continuous HTTP 404 errors.

Rectified Code for malware_simulator.py (lines 104-107):

```
# Set DISPLAY_MODE to 'scanner' for app.py prototype (or 'alerts' for server.py mock):
DISPLAY_MODE = 'scanner'

# Set interval greater than the 33s scan duration to prevent thread pile-up:
REFRESH_INTERVAL = 45.0
```

ERR-07 [MEDIUM]: Windows Console UnicodeEncodeError on Checkmark Character (\u2713 / ✓)

Target Files: detecter.py (Lines 118, 131), make_guide.py (Line 246)

Root Cause: On Microsoft Windows, stdout in cmd.exe/PowerShell defaults to Code Page 1252 (cp1252). The Unicode checkmark character \u2713 ('✓') cannot be mapped to any cp1252 byte value. When a scan completes with 0 threats detected, detecter.py executes print('\n[✓] Scan complete...'), crashing with UnicodeEncodeError: 'charmap' codec can't encode character '\u2713'. make_guide.py has the exact same crash.

Rectified Code:

```
# In detecter.py line 118:
verdict = "[OK] CLEAN"

# In detecter.py line 131:
if threat_count == 0:
print("\n[+] Scan complete: No active fileless injections detected.")

# In make_guide.py line 246:
print("[+] Updated presentation_guide.html with complete master guide!")
```

ERR-08 [LOW]: Kernel Thread Handle Leak in test_injector.py

Target File: test_injector.py (Lines 298-302)

Root Cause: spin_thread = kernel32.CreateThread(...) creates a raw thread handle. In the finally: block, kernel32.TerminateThread(spin_thread, 0) is called, but kernel32.CloseHandle(spin_thread) is omitted, leaving an unclosed kernel handle open in Windows kernel memory.

Rectified Code for test_injector.py (lines 298-303):

```
try:
if kernel32.TerminateThread(spin_thread, 0):
print("[*] Raw spin thread terminated.")
kernel32.CloseHandle(spin_thread) # Close the open kernel handle
except Exception:
pass
```

ERR-09 [MEDIUM]: Dual Divergent Server Architectures (server.py vs app.py)

Target Files: server.py, app.py, index.html, trigger_test_event.py

Root Cause: The repository contains two incompatible backend servers:
1. **Real Engine (app.py):** Runs genuine Win32 VirtualQueryEx memory scanning, scikit-learn ML inference, thread isolation/containment endpoints, and serves an embedded single-page UI at GET /.
2. **Mock Server (server.py):** Synthetic alert generator broadcasting JSON events over WebSocket (/ws/alerts) for the standalone index.html, with a /simulate/alert endpoint.
Because launch_shadowmem.bat runs app.py, running trigger_test_event.py fails with HTTP 404, and index.html fails to connect to WebSockets.

Rectification Recommendation:

- Establish **app.py** as the single authoritative application.
- If WebSocket alerts are desired in the main UI, migrate server.py's /ws/alerts and /simulate/alert routes into app.py.
- Clarify in documentation and scripts that server.py is a mock testing stub.

3. Verification Checklist After Applying Fixes

- [ ] **1. Console Script Check:** Run .\.venv\Scripts\python.exe detecter.py. Ensure it completes with [+] Scan complete without UnicodeEncodeError.
- [ ] **2. Sandbox Test Suite:** Run .\.venv\Scripts\python.exe test_sandbox.py. All 6 test suites must PASS and exit with code 0.
- [ ] **3. Live Scan & Containment Test:** With app.py running, run .\.venv\Scripts\python.exe test_fixes.py. The memory scan must complete within 60s without socket timeouts.
- [ ] **4. Real Process Isolation Test:** Test isolating a non-critical test process via POST /api/isolate/{pid}. Verify that handles are closed properly and no AttributeError is raised.
- [ ] **5. UI Resume Button Test:** Open http://127.0.0.1:8000, scan, inspect an anomaly, click 'Isolate Process'. Confirm the button updates to 'Resume Process' and remains clickable.